OpenCV/Python Integration with Unity3D

Marker-Based Augmented Reality

Computer Vision and Mixed Reality (VARM) — MEIM, ISEL

---

1. Goal

This guide describes how to integrate a Python/OpenCV ArUco marker detection application with the Unity3D game engine, enabling the real-time rendering of sophisticated 3D objects aligned with markers.

The architecture is based on UDP communication between two processes:

- Python/OpenCV — captures the camera feed, detects ArUco markers, estimates the pose, and sends the data to Unity.
- Unity3D — receives the camera image as a background and positions 3D objects aligned with the detected markers.

2. Software Requirements

2.1 Python

Component	Recommended Version
Python (Anaconda)	3.10.x
opencv-contrib-python	4.9.0.80
numpy	1.26.4

> **Note:** *Always use opencv-contrib-python (not opencv-python) to have access to the cv2.aruco module.*

2.2 Unity3D

Component	Version
Unity Hub	Latest version
Unity	6000.0.x LTS (Unity 6.0 LTS)
Render Pipeline	Built-in (not URP)

> **Note:** *When creating the Unity project, select the '3D' template (Built-in), not '3D (URP)'. The Unlit/Texture shader used for the video background is not available in URP.*

3. Application Architecture

Communication between Python and Unity uses UDP on two separate channels:

Channel	Port	Content	Format
Video	5000	Compressed camera frame	Fragmented JPEG (UDP chunks)
Pose	5001	Marker pose data	JSON {id, rvec, tvec}

Structure of the pose JSON sent by Python:

```
{"id":50,"rvec":[[rx,ry,rz]],"tvec":[[tx,ty,tz]]}
```

Note: *UDP has a limit of ~65KB per packet. JPEG frames are fragmented into chunks with a 4-byte header (2 bytes chunk index + 2 bytes total chunks).*

4. Python/OpenCV Application

4.1 Code Structure

The Python code is based on the existing ArUco detection file, with the following additions:

- Import of the socket and json modules.
- Configuration of two UDP sockets (frames and pose).
- A sendFrame() function to send fragmented frames.
- Sending the pose of each detected marker in the main loop.

4.2 Socket Configuration

```
import socket, json
UNITY_IP   = '127.0.0.1'   # same PC; change if Unity is on another machine
PORT_FRAME = 5000          # port for video frames
PORT_POSE  = 5001          # port for pose data
MAX_UDP    = 60000         # safe UDP packet size limit
sock_frame = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock_pose  = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

4.3 Sending Frames (fragmented)

```
def sendFrame(sock, frame, ip, port):
    _, buffer = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 60])
    data = buffer.tobytes()
    num_chunks = (len(data) + MAX_UDP - 1) // MAX_UDP
    for i in range(num_chunks):
        chunk = data[i*MAX_UDP:(i+1)*MAX_UDP]
        header = i.to_bytes(2,'big') + num_chunks.to_bytes(2,'big')
        sock.sendto(header + chunk, (ip, port))
```

4.4 Sending Pose in the Main Loop

```
sendFrame(sock_frame, frame, UNITY_IP, PORT_FRAME)
for rvec, tvec, ids_i in zip(rotation_vectors, translation_vectors, ids):
    pose_data = {'id': int(ids_i), 'rvec': rvec.tolist(), 'tvec': tvec.tolist()}
    msg = json.dumps(pose_data, separators=(',',':')).encode()
    sock_pose.sendto(msg, (UNITY_IP, PORT_POSE))
```

Note: *Use separators=(',',':') in json.dumps() to remove whitespace and simplify parsing in Unity.*

5. Unity3D Application

5.1 Project Configuration

1. Create a new project in Unity Hub using the '3D' template (Built-in Render Pipeline).
2. Create the folder Assets/Scripts for the C# scripts.
3. Configure the Game View with a custom resolution of 640x480 (Fixed Resolution).

5.2 Scene Hierarchy

The scene should have the following structure in the Hierarchy window:

```
SampleScene
├── Main Camera          <- associated with PoseReceiver script
├── Directional Light
├── EventSystem
├── Canvas
│   └── RawImage         <- associated with VideoReceiver script, displays the video
└── ARPivot_XX           <- empty GameObject (pivot) for each marker
    └── ARObject_XX      <- 3D object child of the pivot
```

Note: *ARObjects must NOT be children of the Main Camera. They should be at the root of the Hierarchy or as children of a pivot object.*

5.3 Canvas Configuration (Video Background)

4. Create Canvas: Hierarchy -> right-click -> UI -> Canvas.
5. In the Canvas Inspector, set Render Mode to 'Screen Space - Camera'.
6. Set Render Camera to ‘Main Camera (Camera)’ (drag Main Camera from the Hierarchy).
7. Create RawImage: right-click on Canvas -> UI -> Raw Image.
8. Select the RawImage and set anchors to 'stretch/stretch' (Alt + click on the anchor icon).
9. Add the VideoReceiver script to the RawImage and assign the RawImage itself to the 'Display Image' field.

5.4 AR Objects with Pivot

To make the virtual object appear with its base on the marker (instead of its centre), use an empty parent GameObject as a pivot:

10. Create empty object: right-click in Hierarchy -> Create Empty -> rename to ARPivot_XX.
11. Create 3D object: right-click -> 3D Object -> choose the desired object -> rename to ARObject_XX.
12. Make ARObject_XX a child of ARPivot_XX (drag in the Hierarchy).
13. In the ARObject_XX Inspector, set Position Y = 0.5 (for a Capsule of height 1) so the base sits on the marker.
14. Uncheck the ARPivot_XX checkbox in the Inspector to hide it by default.
15. Ensure the ARObject_XX checkbox is checked.

5.5 Main Camera Configuration

Field	Value
Position	(0, 0, 0)
Rotation	(0, 0, 0)
Clear Flags	Solid Color
Background	Black (#000000)
Near Clip Plane	0.01
Far Clip Plane	1000

Note: *The virtual camera is automatically configured by the PoseReceiver script using the real camera's intrinsic parameters (fx, fy, cx, cy).*

6. VideoReceiver.cs

The VideoReceiver script is responsible for receiving camera frames sent from Python via UDP, reassembling fragmented packets, and displaying the resulting image as the scene background using a Unity RawImage component.

6.1 Frame Fragmentation Protocol

Since UDP limits each packet to approximately 65KB, and a JPEG frame can easily exceed this, the Python side splits each frame into numbered chunks. Each UDP packet sent by Python has a 4-byte header followed by the payload:

Bytes	Content	Description
0–1	Chunk index (big-endian)	Index of this chunk (0-based)
2–3	Total chunks (big-endian)	Total number of chunks for this frame
4–N	JPEG payload	Portion of the compressed frame data

On the Unity side, the ReceiveLoop reads the header, stores each chunk in a dictionary, and reassembles the full frame when all chunks have arrived:

```
// "data" is the byte array read via UDP
int chunkIndex  = (data[0] << 8) | data[1];
int totalChunks = (data[2] << 8) | data[3];
byte[] payload  = new byte[data.Length - 4];
System.Array.Copy(data, 4, payload, 0, payload.Length);

chunks[chunkIndex] = payload;
if (chunks.Count == totalChunks)
{
    // Reassemble all chunks in order
    List<byte> fullFrame = new List<byte>();
    for (int i = 0; i < totalChunks; i++)
        fullFrame.AddRange(chunks[i]);
    latestFrameData = fullFrame.ToArray();
    newFrameAvailable = true;
}
```

6.2 Applying the Frame as a Texture

Once a complete frame is reassembled, it is applied to a Texture2D object and assigned to the RawImage in the Unity main thread (Update), since Unity does not allow texture operations outside the main thread:

```
void Update()
{
    if (newFrameAvailable && latestFrameData != null)
    {
        videoTexture.LoadImage(latestFrameData); // decode JPEG
        videoTexture.Apply();
        newFrameAvailable = false;
    }
}
```

The material is created in code using the Unlit/Texture shader (Built-in pipeline), which renders the texture without any lighting effects — essential for the video background:

```
videoTexture = new Texture2D(imageWidth, imageHeight, TextureFormat.RGB24, false);

Material mat = new Material(Shader.Find("Unlit/Texture"));
mat.mainTexture = videoTexture;
videoRenderer.material = mat;
```

Note: *Always create the material in code rather than assigning it manually in the Inspector. This avoids Unity creating material instances that lose the texture reference at runtime.*

6.3 Inspector Fields

Field	Value	Description
Port	5000	UDP port to receive video frames
Display Image	RawImage	Drag the RawImage from the Hierarchy
Image Width	640	Real camera width in pixels
Image Height	480	Real camera height in pixels

7. PoseReceiver.cs

The PoseReceiver script receives the ArUco marker pose from Python, converts the OpenCV coordinate system to Unity, configures the virtual camera to match the real camera, and positions the AR object accordingly.

7.1 Configuring the Virtual Camera — Projection Matrix

For the virtual object to appear correctly aligned with the real marker in the video, the Unity virtual camera must have exactly the same intrinsic parameters as the real camera used for calibration.

In OpenCV, the camera is modelled by the intrinsic matrix K:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Where:

- f_x, f_y — focal lengths in pixels (how much the lens magnifies along X and Y).
- c_x, c_y — optical centre coordinates (principal point, ideally at the image centre).

Unity uses a projection matrix to transform 3D points into screen coordinates. The equivalent OpenCV-to-Unity projection matrix is built as follows:

$$\begin{aligned}
\text{projMatrix}[0,0] &= 2 * f_x / \text{width} \\
\text{projMatrix}[0,2] &= 1 - 2 * c_x / \text{width} \\
\text{projMatrix}[1,1] &= 2 * f_y / \text{height} \\
\text{projMatrix}[1,2] &= -1 + 2 * c_y / \text{height} \\
\text{projMatrix}[2,2] &= -(\text{far} + \text{near}) / (\text{far} - \text{near}) \\
\text{projMatrix}[2,3] &= -2 * \text{far} * \text{near} / (\text{far} - \text{near}) \\
\text{projMatrix}[3,2] &= -1
\end{aligned}$$

In C# code:

```
float near = cam.nearClipPlane;
float far  = cam.farClipPlane;
float w    = imageWidth;
float h    = imageHeight;

Matrix4x4 projMatrix = Matrix4x4.zero;
projMatrix[0, 0] =  2f * fx / w;
projMatrix[0, 2] =  1f - 2f * cx / w;
projMatrix[1, 1] =  2f * fy / h;
projMatrix[1, 2] = -1f + 2f * cy / h;
projMatrix[2, 2] = -(far + near) / (far - near);
projMatrix[2, 3] = -2f * far * near / (far - near);
projMatrix[3, 2] = -1f;

cam.projectionMatrix = projMatrix;
```

> **Note:** *This matrix replaces Unity's default fieldOfView-based projection. It encodes both the focal lengths and the optical centre offset, ensuring the virtual camera exactly matches the real one.*

To obtain the intrinsic parameters from the calibration file, run the following Python script:

```
import numpy as np
with np.load('webcam_calibration_ouput.npz') as X:
    mtx = X['mtx']
    print(f'fx: {mtx[0,0]:.4f}')
    print(f'fy: {mtx[1,1]:.4f}')
    print(f'cx: {mtx[0,2]:.4f}')
    print(f'cy: {mtx[1,2]:.4f}')
```

7.2 Coordinate System Conversion: OpenCV to Unity

OpenCV and Unity use different coordinate systems. This mismatch must be corrected both for the position (translation vector) and for the rotation.

Axis	OpenCV	Unity	Conversion
X	Points right	Points right	Keep sign
Y	Points down	Points up	Negate
Z	Points forward (into scene)	Points forward (into scene)	Keep sign
Rotation	Rodrigues vector	Quaternion	Convert via rotation matrix

7.2.1 Translation Vector Conversion

The translation vector tvec = [tx, ty, tz] from OpenCV represents the position of the marker origin in the camera coordinate system. The Y axis is simply negated to convert from OpenCV (Y down) to Unity (Y up):

$$\text{position\_Unity} = (tx,\ -ty,\ tz)$$

```
Vector3 position = new Vector3(lat_tx, -lat_ty, lat_tz);
```

7.2.2 Rotation Vector Conversion — Rodrigues to Rotation Matrix

OpenCV represents rotation as a Rodrigues vector rvec = [rx, ry, rz]. This is a compact representation where:

- The direction of the vector gives the rotation axis.
- The magnitude (length) of the vector gives the rotation angle in radians.

$$angle = \|rvec\| = \sqrt{rx^2 + ry^2 + rz^2}$$
$$axis = rvec/angle = (ax, ay, az)$$

The Rodrigues formula converts this axis-angle representation into a 3x3 rotation matrix R:

$$\boldsymbol{R} = \boldsymbol{c} * \boldsymbol{I} + (\boldsymbol{1} - \boldsymbol{c}) * (\boldsymbol{axis} \times \boldsymbol{axis}^T) + \boldsymbol{s} * [\boldsymbol{axis}]_\times$$

Where $c = \cos(angle)$, $s = sin(angle)$, and $[\boldsymbol{axis}]_\times$ is the skew-symmetric matrix. Expanding this gives each element of R:

```
R[0,0] = t*ax*ax + c       R[0,1] = t*ax*ay - s*az    R[0,2] = t*ax*az + s*ay
R[1,0] = t*ax*ay + s*az    R[1,1] = t*ay*ay + c       R[1,2] = t*ay*az - s*ax
R[2,0] = t*ax*az - s*ay    R[2,1] = t*ay*az + s*ax    R[2,2] = t*az*az + c
```

In C# code:

```
float angle = Mathf.Sqrt(lat_rx*lat_rx + lat_ry*lat_ry + lat_rz*lat_rz);
float ax = lat_rx / angle;
float ay = lat_ry / angle;
float az = lat_rz / angle;
float c  = Mathf.Cos(angle);
float s  = Mathf.Sin(angle);
float t  = 1f - c;

float r00 = t*ax*ax + c;     float r01 = t*ax*ay - s*az;  float r02 = t*ax*az + s*ay;
float r10 = t*ax*ay + s*az;  float r11 = t*ay*ay + c;     float r12 = t*ay*az - s*ax;
float r20 = t*ax*az - s*ay;  float r21 = t*ay*az + s*ax;  float r22 = t*az*az + c;
```

7.2.3 Applying the Axis Conversion to the Rotation Matrix

After computing the OpenCV rotation matrix R, the Y-axis sign differences between OpenCV and Unity must be applied. This is done by negating specific rows and columns of R that involve the Y axis:

```
R_Unity[0,0] =  R[0,0]    R_Unity[0,1] = -R[0,1]    R_Unity[0,2] =  R[0,2]
R_Unity[1,0] = -R[1,0]    R_Unity[1,1] =  R[1,1]    R_Unity[1,2] = -R[1,2]
R_Unity[2,0] =  R[2,0]    R_Unity[2,1] = -R[2,1]    R_Unity[2,2] =  R[2,2]
```

In C# code, using a Matrix4x4:

```
Matrix4x4 mat = Matrix4x4.identity;
mat[0,0] =  r00;  mat[0,1] = -r01;  mat[0,2] =  r02;
mat[1,0] = -r10;  mat[1,1] =  r11;  mat[1,2] = -r12;
mat[2,0] =  r20;  mat[2,1] = -r21;  mat[2,2] =  r22;

Quaternion rotation = mat.rotation; // Unity extracts Quaternion from Matrix4x4
```

7.3 Applying Pose to the AR Object

Once the position and rotation are computed in Unity coordinates, they are applied to the AR pivot object. An additional initial rotation can be added to orient the 3D object perpendicular to the marker surface (along the marker's Z axis):

```
// Optional: rotate object to stand perpendicular to the marker
Quaternion initialRotation = Quaternion.Euler(90, 0, 0);

target.transform.position = position;
target.transform.rotation = rotation * initialRotation;
```

The multiplication rotation * initialRotation first applies the initial orientation of the object, then applies the marker's pose — ensuring the object is correctly oriented relative to the marker at all times.

7.4 Pivot Object for Base Alignment

By default, Unity positions and rotates objects around their geometric centre. To make the base of the object sit on the marker instead of its centre, a pivot pattern is used:

16. Create an empty GameObject (ARPivot_XX) — this is the object the script moves and rotates.
17. Make the 3D object (ARObject_XX) a child of the pivot.
18. Offset the child's local position so its base aligns with the pivot origin.

For a Capsule of height 1 unit, the base is 0.5 units below the centre. Setting the child's local position to Y = 0.5 moves it upward so the base sits at the pivot origin (the marker centre):

```
// In the Inspector, for ARObject_XX (child of ARPivot_XX):
Position: (0, 0.5, 0)   // half the object height
```

Note: *The pivot offset value depends on the object's height. For a default Unity Capsule (height = 1), Y = 0.5 is correct. Adjust for other object sizes.*

7.5 Marker Visibility Timeout

When the marker leaves the camera's field of view, the Python side stops sending pose data. A timeout mechanism hides the AR object after a configurable delay:

```
if (Time.time - lastPoseTime > markerTimeout)
{
    HideAllObjects();
}
```

7.6 Inspector Fields

Field	Value	Description
Port	5001	UDP port to receive pose data
Ar Object XX	ARPivot_XX	Drag the pivot for each marker ID
Fx	from calibration	Focal length X (pixels)
Fy	from calibration	Focal length Y (pixels)
Cx	from calibration	Optical centre X (pixels)
Cy	from calibration	Optical centre Y (pixels)
Image Width	640	Camera resolution width
Image Height	480	Camera resolution height
Marker Timeout	0.3	Seconds before hiding the object when marker is lost

8. Execution Procedure

19. Ensure Unity is in Play mode (click the Play button).
20. Run the Python script with the camera connected.
21. The camera video appears as the background in the Unity Game View.
22. When an ArUco marker is shown to the camera, the associated 3D object appears over the marker.
23. When the marker leaves the field of view, the object disappears after the configured timeout.

Note: *Always start Unity first, then run Python. If Python is started before Unity, the initial UDP packets will be lost.*

9. Common Issues and Solutions

Problem	Likely Cause	Solution
Video does not appear in Unity	Shader incompatible with render pipeline	Ensure the project uses Built-in (not URP); shader must be Unlit/Texture
Virtual object fixed at centre	Incorrect intrinsic parameters	Fill in fx, fy, cx, cy with real calibration values in the Inspector
Object appears behind the video	Canvas in front of 3D objects	Use Canvas in Screen Space - Overlay mode
UDP not reaching Unity	Firewall blocking the port	Allow Unity through Windows Defender Firewall
Error: ambiguous reference Debug	Namespace conflict	Add: using Debug = UnityEngine.Debug;
Object does not disappear	Marker Timeout too high	Reduce the Marker Timeout value in the Inspector
Base of object not on marker	Pivot offset not configured	Set Position Y = 0.5 on the ARObject child of the pivot
Object jitters when marker is still	UDP packet loss or network delay	Reduce JPEG quality or add a smoothing filter on position/rotation